Comprehensive Guide to Secrets Storage and Management

Programmatic approaches to securely storing, using, and managing secrets — locally and remotely

---

Introduction

Every non-trivial application depends on secrets: API keys, database credentials, encryption keys, tokens, certificates, and signing keys. How you store, access, rotate, and protect these secrets determines a large part of your security posture. A leaked database password or a committed AWS key can cascade into a full-scale breach within minutes.

This guide surveys the full landscape of available options for programmatic secrets management — from a single encrypted file on a developer's laptop to globally distributed threshold key management systems. It also explains how **envelope encryption** serves as the unifying architectural pattern that ties most of these approaches together.

The Envelope Encryption Pattern

Before diving into specific solutions, it's worth understanding the pattern that underpins nearly all of them.

In envelope encryption, each secret (or logical group of secrets) is encrypted with its own **Data Encryption Key (DEK)**. The DEK itself is then encrypted by a **Key Encryption Key (KEK)**. The KEK is the single root secret that requires the strongest protection — an HSM, a TPM, a cloud KMS, or a hardware token.

This architecture offers several practical advantages. You can store millions of encrypted secrets in any convenient medium (SQLite, S3, a flat file) because the ciphertext is useless without the DEK, and the DEK is useless without the KEK. Rotating the KEK requires only re-encrypting the DEKs (small, fast operations), not re-encrypting all the underlying data. And compromising a single DEK exposes only the data it protects, not everything in the system.

Nearly every solution described below either implements envelope encryption internally or can serve as a building block within an envelope encryption architecture.

---

1. Local File and Database Storage with Application-Level Encryption

The simplest approach: encrypt secrets with a symmetric key (typically AES-256-GCM) and store the ciphertext locally in SQLite, LevelDB, BoltDB, or even flat files (JSON, YAML, TOML). The encryption key is injected at runtime through an environment variable, a CLI argument, or read from a separate protected file.

This works well for single-machine deployments, CLI tools, and embedded applications where external dependencies are undesirable. The main challenge is protecting the encryption key itself — it has to come from somewhere, and if it's in an environment variable, it's visible to anyone who can inspect the process environment.

Encrypted Configuration File Tools

Several tools formalize this pattern:

- **SOPS** (Secrets OPerationS, originally by Mozilla, now a CNCF project) encrypts individual values within YAML, JSON, INI, and ENV files while leaving keys in plaintext. This makes files diffable, version-controllable, and human-readable in structure. SOPS supports multiple KEK backends: AWS KMS, GCP KMS, Azure Key Vault, HashiCorp Vault Transit, age, and PGP. You can require multiple KEK sources simultaneously (e.g., both a team PGP key and an AWS KMS key), providing defense in depth.
- **age** is a modern, minimal file encryption tool designed as a replacement for PGP in many use cases. It uses X25519 for key exchange and ChaCha20-Poly1305 for symmetric encryption. Its simplicity makes it attractive for scripted workflows and as a SOPS backend.
- **GPG/PGP** remains widely used for encrypting secrets files, particularly in combination with tools like `pass` (the Unix password manager, which stores each secret as a GPG-encrypted file in a directory tree) and `gopass` (a team-oriented rewrite).
- **git-crypt** applies transparent, selective encryption to specific files in a Git repository. Files are encrypted on commit and decrypted on checkout, using either GPG keys or a shared symmetric key.
- **dotenvx** encrypts `.env` files with a project-level key, keeping the developer ergonomics of `.env` while adding encryption at rest.

---

2. Operating System Keyrings and Credential Managers

Every major operating system provides a protected credential store accessible through standardized APIs. These stores typically benefit from OS-level protections: access control tied to user sessions, encryption at rest using platform-specific mechanisms, and (on some platforms) hardware-backed key protection.

Linux

The **freedesktop.org Secret Service specification** defines a D-Bus API for storing and retrieving secrets. Two major implementations exist:

- **GNOME Keyring** — the default on GNOME-based desktops, accessible via D-Bus and the `libsecret` library.
- **KDE Wallet (KWallet)** — the default on KDE Plasma desktops, also accessible via D-Bus.

The `libsecret` C library (and its bindings for Python, Vala, JavaScript, etc.) provides a unified programmatic interface that works with either backend. For headless servers without a desktop environment, `gnome-keyring-daemon` can run in a minimal mode, or you can use **KeePassXC** with its Secret Service integration.

macOS

Keychain Services provides hardware-backed secure storage. On Macs with a Secure Enclave (T2 chip or Apple Silicon), keychain items can be protected by hardware. Programmatic access is through the `Security.framework` APIs (`SecItemAdd`, `SecItemCopyMatching`, etc.) or the `security` command-line tool. The keychain supports passwords, keys, certificates, and arbitrary data blobs.

Windows

Two primary mechanisms exist:

- **Credential Manager** — accessed via the `CredRead`/`CredWrite` Win32 API functions. Stores credentials associated with target names, scoped to the current user.
- **Data Protection API (DPAPI)** — encrypts arbitrary data blobs using keys derived from the user's logon credentials. This is particularly useful because it's machine- and user-bound without requiring you to manage keys at all. `CryptProtectData`/`CryptUnprotectData` are the core API calls.

Cross-Platform Libraries

- **Python** `keyring` — abstracts over macOS Keychain, Windows Credential Manager, and Linux Secret Service.
- `keytar` (npm) — Node.js native module providing the same cross-platform abstraction.
- `go-keyring` (Go) — similar abstraction for Go applications.

---

3. Hardware Security Modules (HSMs)

HSMs are dedicated, tamper-resistant hardware devices that generate, store, and use cryptographic keys without ever exposing the raw key material to software. If an attacker compromises the host operating system, they still cannot extract the keys — they can only ask the HSM to perform operations, and the HSM enforces its own access policies.

HSMs are the gold standard for protecting root keys (KEKs) in an envelope encryption scheme.

Physical and Network-Attached HSMs

Enterprise-grade HSMs from vendors like **Thales Luna** (formerly SafeNet/Gemalto), **Entrust nShield**, **Utimaco**, and **Futurex** connect via PCIe or network interfaces and are standard in banking, PKI infrastructure, certificate authorities, and government environments. They're

certified to standards like FIPS 140-2/140-3 Level 3 (which includes physical tamper-response mechanisms).

Cloud HSMs

Cloud providers offer managed HSM services that provide dedicated, single-tenant HSM instances:

- **AWS CloudHSM** — dedicated HSMs within your VPC, FIPS 140-2 Level 3 certified.
- **Google Cloud HSM** — HSM-backed keys within Cloud KMS, accessible via PKCS#11.
- **Azure Dedicated HSM** — Thales Luna HSMs deployed in Azure data centers.

These expose PKCS#11 interfaces, so you can use the same tooling and code as with on-premises HSMs.

Software HSMs

SoftHSM (by the OpenDNSSEC project) provides a PKCS#11-compatible interface implemented entirely in software. It stores keys in an encrypted database on disk. While it doesn't provide the physical tamper resistance of a real HSM, it's invaluable for development, testing, and CI/CD environments. PKCS#11 defines an abstraction between an application and a security module — if your code is written against the PKCS#11 API, you can swap SoftHSM for a real HSM in production without code changes.

The PKCS#11 Standard

PKCS#11 (Cryptoki) is the standard C API for interacting with cryptographic tokens. It provides a uniform interface regardless of whether the underlying device is a physical HSM, a cloud HSM, a TPM, a smart card, or a software emulation. Libraries like **crypto11** (Go), **python-pkcs11**, and **pkcs11-tool** (OpenSC) make it accessible from various programming languages.

---

4. Trusted Platform Modules (TPMs)

TPMs are hardware chips (TPM 1.2 or TPM 2.0) embedded in most modern motherboards, laptops, and servers. Unlike HSMs, which are separate dedicated devices, TPMs are integrated into the platform itself.

A TPM's killer feature is **sealing**: it can encrypt data bound to the platform's specific state, represented by Platform Configuration Registers (PCRs). PCRs contain measurements of the boot process — firmware, bootloader, kernel, initrd. A sealed secret can only be decrypted when the machine boots in a known-good configuration. If someone tampers with the boot chain, the PCR values change and the TPM refuses to unseal.

This is the mechanism behind full-disk encryption in **BitLocker** (Windows), **LUKS with systemd-cryptenroll** (Linux), and **FileVault** (macOS, via the Secure Enclave which serves a similar role).

For secrets management, TPMs are especially useful for protecting local root keys. You seal your KEK to the TPM, and it can only be unsealed on that specific machine in that specific boot state. The `tpm2-pkcs11` project provides a PKCS#11 interface for TPM2 hardware, enabling applications that already support PKCS#11 to use a TPM transparently.

Programmatic access is through:

- **tpm2-tools** — command-line utilities for TPM2 operations.
- **tpm2-tss** — the TCG Software Stack, the low-level C library.
- **go-tpm** — Go library for direct TPM interaction.
- **tpm2-pkcs11** — PKCS#11 provider backed by a TPM.

---

5. Smart Cards and Hardware Security Tokens

Devices like **YubiKeys**, **Nitrokeys**, **SoloKeys**, and traditional **smart cards** (PIV, OpenPGP) can store private keys and perform cryptographic operations on-device. The key never leaves the hardware.

These devices are typically associated with user authentication (FIDO2/WebAuthn, SSH keys, GPG signing), but they can also protect encryption keys for secrets management. You store your KEK on the hardware token and require physical interaction (touch) or a PIN to decrypt. This adds a strong human-in-the-loop requirement that makes automated exfiltration much harder.

Relevant interfaces and standards:

- **PIV (Personal Identity Verification)** — NIST standard for smart card authentication, supported by YubiKey.
- **OpenPGP card specification** — allows GPG operations to be performed on the card.
- **FIDO2/CTAP** — primarily for authentication, but the underlying key storage is general-purpose.
- **PKCS#11** — most smart cards provide a PKCS#11 module.

The limitation is obvious: hardware tokens require physical presence or network-attached readers, making them unsuitable for fully automated, unattended workloads. They're best suited for protecting the root of a key hierarchy where human authorization is acceptable.

---

6. Cloud Key Management Services (KMS)

Cloud KMS services are managed key management backends where the cloud provider holds keys in their HSM infrastructure. You never see the raw key material — you call APIs to encrypt, decrypt, sign, and verify.

AWS KMS

AWS Key Management Service manages Customer Master Keys (CMKs) backed by FIPS 140-2 validated HSMs. It integrates natively with virtually every AWS service (S3, EBS, RDS, Lambda, etc.) and supports automatic key rotation, key policies, and grants for fine-grained access control. The Encrypt/Decrypt/GenerateDataKey APIs are the foundation of envelope encryption across AWS.

Google Cloud KMS

Google Cloud KMS provides key management with software-backed, HSM-backed, and external key options. It integrates with Cloud IAM for access control and supports automatic and manual key rotation. The Cloud External Key Manager (EKM) allows using keys managed by a third-party key manager outside Google's infrastructure.

Azure Key Vault

Azure Key Vault manages keys, secrets, and certificates with HSM-backed tiers available. It integrates with Azure Active Directory for access control and supports soft-delete and purge protection for key recovery.

Cloud Secrets Managers (Higher-Level Abstraction)

Beyond raw KMS encryption, each cloud offers a secrets-specific service that handles the full lifecycle — storage, rotation, access control, and audit logging — with envelope encryption built in:

- **AWS Secrets Manager** — stores and rotates database credentials, API keys, and other secrets with native Lambda-based rotation.
- **GCP Secret Manager** — provides secret storage with versioning, access controls, and audit logs.
- **Azure Key Vault Secrets** — manages secrets alongside keys and certificates in a unified service.

---

7. Self-Hosted Secrets Management Platforms

For organizations that need secrets management across infrastructure but want to maintain full control (or operate in air-gapped environments), several self-hosted platforms exist.

HashiCorp Vault

Vault is the most widely deployed self-hosted secrets manager. It offers dynamic secrets (generating short-lived credentials for databases, cloud providers, PKI, etc.), a transit encryption engine (encrypt/decrypt as a service without storing the data), multiple authentication backends (LDAP, OIDC, Kubernetes, AWS IAM, etc.), and comprehensive audit logging.

Vault uses an unsealing process where the master key is split using Shamir's Secret Sharing — multiple key holders must cooperate to unseal a Vault instance. The master key can also be auto-unsealed using a cloud KMS, an HSM via PKCS#11, or a transit seal from another Vault instance.

The trade-off is operational complexity: Vault requires careful deployment, monitoring, backup, and upgrade procedures. It's licensed under the Business Source License (BSL) since 2023.

OpenBao

After HashiCorp's license change, the **OpenBao** project forked Vault under the Mozilla Public License 2.0. It aims to maintain API compatibility while remaining fully open source.

Infisical

An open-source secrets management platform focused on developer experience. It provides end-to-end encrypted secret storage, environment-based organization, automatic integrations with CI/CD and infrastructure tools (Docker, Kubernetes, Terraform, GitHub Actions, Vercel), a CLI for injecting secrets into processes, and secret scanning for leak prevention.

CyberArk Conjur

Targets regulated enterprises with deep integration into CyberArk's broader Privileged Access Management (PAM) suite. It provides machine identity authentication, fine-grained policy-based access control, and session monitoring. Common in financial services, healthcare, and government.

Other Self-Hosted Options

- **Barbican** — OpenStack's key management service, designed for managing secrets within OpenStack deployments.
- **Keywhiz** — Square's open-source secret management system, focused on distributing secrets to services.
- **Knox** — Pinterest's open-source secret management service.
- **Confidant** — Lyft's open-source secrets management service, using AWS KMS and DynamoDB.

---

8. SaaS / Cloud-Native Secrets Platforms

For teams that want secrets management without the operational burden of self-hosting.

Doppler

A SaaS-first secrets management platform emphasizing developer experience. It provides a universal secrets dashboard, automatic syncing across environments, CLI-based secret injection, and integrations with major CI/CD platforms and cloud providers. Secrets are versioned and access is logged.

Akeyless

Uses a patented **Distributed Fragments Cryptography (DFC)** approach: the encryption key is split into fragments, with the customer always controlling one fragment. This means even Akeyless cannot decrypt customer secrets — a zero-knowledge architecture delivered as SaaS. It supports dynamic secrets, just-in-time access, automatic rotation, and a gateway component that runs in the customer's environment for fallback and caching.

1Password Secrets Automation

1Password has expanded from consumer password management into developer secrets management. It offers a Secrets Automation platform with Connect servers (self-hosted proxies to 1Password vaults), CLI tools, and SDK integrations. The security model inherits 1Password's established end-to-end encryption.

Bitwarden Secrets Manager

Bitwarden provides cost-effective secrets management designed for development teams, including machine account support for CI/CD workflows, CLI and SDK integration across major languages, and event logging with audit trails. Its open-source heritage provides transparency into the security model.

Keeper Secrets Manager

Extends Keeper's enterprise password management into DevOps secrets, with a zero-knowledge security model, CI/CD integration, and detailed analytics.

StrongDM

Takes a vault-agnostic approach, providing a unified access layer that works with multiple underlying secret stores. It focuses on Zero Trust access controls, policy-based credential rotation, and comprehensive audit logging for compliance.

---

9. CI/CD and Platform-Native Secret Stores

Most development platforms include built-in secrets management, typically serving as the "last mile" delivery mechanism:

- **GitHub Actions Secrets** — repository, environment, and organization-level secrets injected as environment variables during workflow runs.
- **GitLab CI/CD Variables** — project and group-level variables with optional masking and environment scoping.
- **Bitbucket Pipelines Secure Variables** — encrypted variables available during pipeline execution.
- **Jenkins Credentials** — stores credentials (passwords, SSH keys, tokens, certificates) with plugin-based backends.

- **CircleCI Contexts** — shared environment variables scoped to organizations and security groups.
- **AWS Systems Manager Parameter Store** — hierarchical key-value store with KMS encryption, often used for configuration and secrets in AWS environments.

Container and Orchestration Platforms

- **Kubernetes Secrets** — base64-encoded by default (not encrypted), but can be backed by external stores via the **Secrets Store CSI Driver** (mounts secrets from external vaults as volumes) or the **External Secrets Operator** (syncs secrets from external stores into Kubernetes Secret objects). Kubernetes also supports **encryption at rest** for etcd using envelope encryption with a KMS provider.
- **Docker Swarm Secrets** — secrets are encrypted in transit and at rest in the Raft log, mounted as in-memory files in containers.
- **Nomad Variables** — HashiCorp Nomad's built-in encrypted key-value store for workload secrets.

---

10. Infrastructure-as-Code and Configuration Management

When secrets must live alongside infrastructure definitions, these tools provide in-place encryption:

- **Ansible Vault** — encrypts entire files or individual variables within playbooks using AES-256. The vault password can be provided interactively, via a file, or through a script that fetches it from an external source.
- **Chef Encrypted Data Bags** — JSON data bags where values are encrypted with a shared secret or per-environment keys.
- **Puppet Hiera eyaml** — encrypts individual values within Hiera YAML files using PKCS7 (asymmetric encryption), allowing plaintext keys with encrypted values.
- **Terraform** — doesn't have built-in encryption, but integrates with Vault, AWS Secrets Manager, and other backends via providers. Sensitive values can be marked to prevent them from appearing in plan output and state. The state file itself should be stored in an encrypted backend (S3 with KMS, Terraform Cloud, etc.).
- **Pulumi** — supports encrypted secrets natively in state, with pluggable encryption providers (Pulumi Service, AWS KMS, GCP KMS, Azure Key Vault, passphrase-based).

---

11. Distributed and Threshold-Based Key Management

In threshold cryptography, no single party holds a complete key. Key shares are distributed

across multiple nodes or parties, and a configurable threshold (e.g., 3 of 5) must cooperate to perform any cryptographic operation.

Shamir's Secret Sharing (SSS)

The foundational primitive. A secret is split into *n* shares such that any *k* shares can reconstruct it, but *k-1* shares reveal nothing. HashiCorp Vault uses SSS for its unseal process. The limitation is that reconstruction happens in one place — the reconstructed secret exists in memory on a single machine, which is a potential vulnerability.

Multi-Party Computation (MPC)

MPC protocols allow parties to jointly compute a function (e.g., signing, decryption) over their individual key shares without ever reconstructing the full key. The key never exists in a single location, even during use. This is cryptographically stronger than SSS for active operations.

Commercial MPC-based key management solutions include:

- **Fireblocks** — MPC-based key management primarily for digital assets, but the underlying technology is general-purpose.
- **Blockdaemon (formerly Sepior)** — threshold MPC for key management and signing.
- **Sodot** — MPC key management with sub-second signing performance.
- **Zama Threshold KMS (dKMS)** — distributed key management using threshold cryptography, designed for decentralized applications.
- **Unbound Security (now Coinbase)** — virtual HSM using MPC, providing HSM-grade key protection in software.

Threshold Signature Schemes (TSS)

Specifically for signing operations, TSS protocols (like GG18, GG20, CGGMP) allow distributed signing without reconstructing the private key. These are widely used in cryptocurrency custody but applicable to any signing use case.

---

12. Encrypted Environment and Secret Injection Tools

These tools bridge the gap between developer ergonomics and security by managing the injection of secrets into application environments:

- **Chamber** (by Segment) — reads and writes secrets to AWS SSM Parameter Store and injects them as environment variables when running commands.
- **envchain** — stores secrets in the OS keychain (macOS Keychain, GNOME Keyring) and exports them as environment variables on demand.
- **direnv** — loads and unloads environment variables based on the current directory. Not a secrets tool per se, but commonly combined with encrypted `.envrc` files.

- **aws-vault** — stores AWS credentials in the OS keychain and generates temporary STS credentials, avoiding long-lived access keys on disk.
- **saml2aws**, **granted** — similar patterns for federated cloud access.

---

13. Secrets Detection and Leak Prevention

Not storage mechanisms themselves, but critical to the overall management lifecycle. These tools catch secrets that have escaped their intended storage:

- **GitGuardian** — monitors repositories (GitHub, GitLab, Bitbucket, private Git servers) in real time, detecting over 350 types of secrets. Offers governance features including honeytokens and Non-Human Identity (NHI) management.
- **TruffleHog** — scans Git repositories, S3 buckets, file systems, and other sources for high-entropy strings and known secret patterns.
- **Gitleaks** — fast, lightweight Git secret scanner with pre-commit hook support.
- **detect-secrets** (by Yelp) — a tool for detecting secrets in codebases, with a baseline file for managing known exceptions.
- **Talisman** (by Thoughtworks) — pre-push Git hook that prevents secrets from being committed.

Comparison Matrix

Category	Examples	Key Protection	Automation Level	Best For
Local encrypted files	SOPS, age, git-crypt	Depends on KEK backend	Medium	Small teams, single-repo projects
OS keyrings	GNOME Keyring, macOS Keychain, DPAPI	OS-level, some HW-backed	Low-Medium	Desktop apps, CLI tools, developer machines
HSMs	Thales Luna, AWS CloudHSM, SoftHSM	Hardware tamper-resistant	Medium	Root key protection, PKI, compliance
TPMs	tpm2-tools, tpm2-pkcs11	Hardware, platform-bound	Medium	Server root keys, disk encryption, attestation
Smart cards/tokens	YubiKey, Nitrokey	Hardware, portable	Low	Human-authorized operations, GPG/SSH keys

Category	Examples	Key Protection	Automation Level	Best For
Cloud KMS	AWS KMS, GCP Cloud KMS, Azure Key Vault	Provider HSM-backed	High	Cloud-native apps, envelope encryption KEK
Cloud secrets managers	AWS Secrets Manager, GCP Secret Manager	Provider-managed	High	Cloud-native apps needing rotation and lifecycle
Self-hosted platforms	Vault, Infisical, Conjur	Configurable	High	Multi-cloud, hybrid, air-gapped environments
SaaS platforms	Doppler, Akeyless, 1Password	Provider-managed	High	Teams wanting minimal operational overhead
CI/CD native	GitHub Secrets, GitLab Variables	Platform-managed	High	Pipeline-scoped secrets
Container platforms	K8s Secrets + CSI Driver, Docker Secrets	Varies by backend	High	Container orchestration environments
IaC tools	Ansible Vault, SOPS, Pulumi	Configurable	Medium	Secrets embedded in infrastructure definitions
Threshold/MPC	Fireblocks, Zama dKMS, SSS	Distributed, no single point	Medium-High	High-security, multi-party custody
Injection tools	Chamber, envchain, aws-vault	Delegates to backend	Medium	Developer workflow, credential brokering
Detection tools	GitGuardian, TruffleHog, Gitleaks	N/A (detection only)	High	Leak prevention, compliance scanning

Architectural Recommendations

For a Single Application on a Single Server

Use the OS keyring or a TPM-sealed key as your KEK. Encrypt secrets with a per-secret DEK stored in SQLite. This gives you hardware-bound key protection with no external dependencies.

For a Small Team with a Few Services

SOPS with a cloud KMS backend (or age keys distributed to team members) provides encrypted-at-rest configuration files that can be version-controlled alongside code. Combine with a CI/CD platform's native secrets for deployment.

For a Multi-Service, Multi-Environment Organization

A dedicated secrets management platform (Vault, Infisical, Doppler, or a cloud-native secrets manager) becomes essential. Use dynamic secrets where possible (short-lived database credentials, temporary cloud tokens) to minimize the blast radius of any single compromise. Back the platform's root keys with an HSM or cloud KMS.

For Regulated Industries or Maximum Security

Layer HSMs or TPMs at the root, a secrets management platform (Vault with HSM auto-unseal, or CyberArk Conjur) for operations, and threshold cryptography or MPC for the most critical keys (signing keys, root CA keys). Add GitGuardian or similar for continuous scanning. Implement comprehensive audit logging at every layer.

Conclusion

The landscape of secrets management is broad, ranging from a simple encrypted file to globally distributed threshold key management. No single solution fits every use case. The key insight is that these tools are composable — you can (and often should) combine several layers. A TPM protects the local root key, a cloud KMS provides the envelope encryption KEK, a secrets manager handles lifecycle and rotation, and a detection tool catches anything that slips through the cracks.

The unifying principle across all approaches is **envelope encryption**: protect many secrets with many DEKs, protect the DEKs with a single KEK, and invest your strongest security measures in protecting that KEK. Where and how you protect the KEK — that's the decision this guide helps you make.